

Semantic Correspondence

<https://github.com/aexomir/semantic-correspondence>

Parsa Tavakoli
s346279

Politecnico di Torino

parsa.tavakoli@studenti.polito.it

AmirHossein Ranjbar
s340808

Politecnico di Torino

amirhossein.ranjbar@studenti.polito.it

Arman Homafar
s338312

Politecnico di Torino

arman.homafar@studenti.polito.it

Abstract

*We study dense semantic correspondence with self-supervised vision backbones, first as frozen feature extractors and then after limited task-specific adaptation. The pipeline begins with a training-free cosine-similarity baseline. From there, we selectively unfreeze the last N transformer blocks of DINOv2 ViT-L/14, DINOv3 ViT-B/16, and SAM ViT-B, optimizing them on SPair-71k with an InfoNCE-style contrastive objective. We also add a window soft-argmax decoding stage, which sharpens the discrete argmax prediction into a sub-patch estimate without introducing any extra training. As a final check, we evaluate zero-shot transfer to PF-Pascal and PF-Willow, where the adapted models show strong cross-dataset generalization. On SPair-71k, the best configuration, DINOv3 with $N=2$ fine-tuned blocks and soft-argmax decoding, reaches **76.41% PCK@0.1**, improving the frozen baseline by **+22.84 pp**. On PF-Willow, DINOv3 obtains **89.22% PCK@0.1** in the zero-shot setting. All code and precomputed results are publicly available at github.com/aexomir/semantic-correspondence.*

1. Introduction

Semantic correspondence asks a deceptively simple question: where does the same semantic part appear in another image from the same object category? A point on the wing of one aeroplane should map to the corresponding wing region of another aeroplane; a keypoint on a cat’s ear should remain meaningful even when the target cat has a different pose, texture, scale, or background. This problem sits between

low-level matching and high-level recognition. It requires local precision, because keypoints are evaluated spatially, but it also requires semantic abstraction, because the two images rarely contain the same physical instance.

That combination makes semantic correspondence useful, and difficult. Reliable dense matches can support pose estimation, image editing, image-guided generation, object analysis, and visual question answering. At the same time, the task exposes a weakness in many visual representations: features that work well for classification, segmentation, or retrieval are not necessarily good at matching small semantic parts across different object instances. Ordinary geometric matching is often allowed to assume similar texture or a nearly shared viewpoint. Semantic correspondence is less forgiving. It must handle articulation, occlusion, intra-class variation, cluttered scenes, and ambiguous object parts, all while producing a location rather than just a category label.

Early systems relied on hand-crafted descriptors such as SIFT [5] and HoG. These descriptors remain valuable in controlled geometric settings, but they are not robust enough for the appearance shifts that define the semantic regime. Deep convolutional features improved the situation substantially, especially when combined with cost volumes, hyperpixel aggregation, or explicit matching modules. More recently, attention has moved toward large self-supervised vision transformers (ViTs). Their patch-level tokens often carry a surprisingly strong part-level organization, even though the models are not trained with keypoint annotations. This makes them attractive as general-purpose correspondence backbones.

However, strong frozen features do not fully answer the problem. A pretrained backbone may encode broad semantic structure, yet still place the best match on a nearby patch,

confuse symmetric parts, or fail on categories whose geometry differs from the pretraining distribution. The central question, then, is not only whether models such as DINOv2, DINOv3, and SAM contain useful correspondence features, but how far those features can be pushed with a small, controlled amount of task-specific adaptation. In this work, we deliberately keep the adaptation lightweight. We unfreeze only the last few transformer blocks and train them with an InfoNCE-style correspondence objective, so that the experiment measures targeted representational alignment rather than full-scale retraining.

We also separate feature learning from decoding. Standard argmax matching selects the single most similar target patch, which is simple and stable, but it is limited by the feature-grid resolution. A correct semantic match can still be penalized if the predicted point is stuck at the center of a coarse patch. To address this, we add a post-hoc window soft-argmax step: after finding the strongest local response, the method computes a continuous estimate inside a small neighborhood. This does not change the backbone and requires no extra training, but it can improve localization when the evaluation threshold is tight.

This work is organized around three concrete questions:

1. How well do modern self-supervised backbones (DINOv2, DINOv3, SAM) perform as *frozen* feature extractors for semantic correspondence on SPair-71k?
2. How much does light fine-tuning of the last N transformer blocks with an InfoNCE loss improve performance, and how does the optimal N vary across architectures?
3. Can a simple *window soft-argmax* decoder, applied post-hoc without retraining, further improve localization accuracy, and do the resulting features transfer to unseen benchmarks (PF-Pascal, PF-Willow)?

The main contributions are as follows:

- A reproducible, single-notebook pipeline for semantic keypoint correspondence on SPair-71k, with support for three backbone families.
- An empirical comparison of frozen vs. fine-tuned backbones under controlled conditions (same loss, same schedule, same evaluation split).
- A window soft-argmax decoding module that achieves consistent gains at tight PCK thresholds with negligible overhead.
- A cross-dataset generalization study on PF-Pascal and PF-Willow that quantifies how SPair-71k fine-tuning affects transfer to the Proposal Flow family.

2. Related Work

Semantic correspondence. Early semantic correspondence methods used hand-crafted features [3] or CNN descriptors coupled with geometric constraints. Later approaches pushed the field forward with hyperpixel pooling

(HPF [6]), cost-volume filtering (CATs [2]), and transformer-based matching architectures such as MatchFormer. The common goal is the same: obtain reliable matches even when two images differ substantially in appearance, viewpoint, or object instance.

Self-supervised visual representations. DINO [1] showed that ViT patch tokens trained through self-distillation can develop emergent semantic segmentation behavior. DINOv2 [8] scaled this idea with curated LVD-142M data and delivered strong out-of-the-box results across many vision tasks. SAM [4], although designed for promptable segmentation rather than correspondence, is also relevant because its image encoder produces dense high-resolution features.

Feature matching with transformers. SD-DINO [10] combines Stable Diffusion with DINOv2 features for correspondence, while DIFT [9] uses activations from diffusion models directly. Our focus is narrower. We study the ViT backbone itself, without a diffusion model or a large retrieval index, so that the effects of light fine-tuning and decoding can be isolated cleanly.

PCK evaluation. Percentage of Correct Keypoints (PCK) at threshold α was popularized by the Proposal Flow benchmark [3]. SPair-71k [7] adopts the same basic idea, using bounding-box normalization to decide whether a predicted keypoint is close enough to the ground truth.

3. Method

3.1. Feature Extraction

Given an image $I \in \mathbb{R}^{3 \times H \times W}$, each backbone produces a spatial feature tensor $F \in \mathbb{R}^{D \times H_f \times W_f}$:

- **DINOv2 ViT-L/14.** Input resized to 518×518 , patch size 14. We extract the `x_norm_patchtokens` output and reshape to a $\approx 37 \times 37$ grid ($D=1024$).
- **DINOv3 ViT-B/16.** Input resized to 512×512 , patch size 16. Patch tokens give a 32×32 feature grid ($D=768$).
- **SAM ViT-B.** Input at native 1024×1024 (no positional embedding interpolation). The image encoder outputs a dense feature map ($D=256$) at 64×64 spatial resolution.

Before matching, all feature maps are **L2-normalized** along the channel dimension. We precompute the features and cache them in float16, which avoids repeated backbone forward passes during evaluation and fine-tuning sweeps.

3.2. Baseline Matching (Stage 1)

For a source keypoint at pixel location (s_x, s_y) , we map to feature coordinates (f_x, f_y) and retrieve the source descriptor $\mathbf{d} = F_{\text{src}}[:, f_y, f_x] \in \mathbb{R}^D$. The predicted target location

is:

$$(\hat{x}, \hat{y}) = \operatorname{argmax}_{h,w} \langle \mathbf{d}, F_{\text{trg}}[:, h, w] \rangle. \quad (1)$$

Feature coordinates are mapped back to pixel space using the source/target image dimensions. This frozen baseline, with no training and $N=0$, provides a direct estimate of the intrinsic correspondence quality of each pretrained backbone.

3.3. InfoNCE Fine-Tuning (Stage 2)

Fine-tuning is deliberately restricted. We update only the last N transformer blocks, together with any normalization or neck layers, and leave the earlier blocks frozen. For each annotated keypoint pair (s_k, t_k) in a training image pair $(I_{\text{src}}, I_{\text{trg}})$, the correspondence objective is formulated as a dense contrastive classification problem over the target feature grid. The source keypoint descriptor is encouraged to assign the highest similarity to the descriptor at the ground-truth target keypoint, while all other $H_f \times W_f$ target locations act as negatives in the same image. Similarities are scaled by a temperature of $\tau=0.07$, so the optimization rewards sharp target peaks rather than merely increasing broad semantic affinity. This setup gives the model a direct keypoint-level alignment signal while keeping the adaptation small enough to preserve the pretrained representation.

Training uses AdamW with learning rate 5×10^{-6} , 3 epochs, batch size 1, gradient accumulation over 4 steps (effective batch size 4), and automatic mixed precision (AMP). Checkpoints are selected by validation PCK@0.1 on SPair-71k val. We sweep $N \in \{1, 2, 4\}$ for each backbone.

3.4. Window Soft-Argmax Decoding (Stage 3)

Once the coarse peak cell $(\hat{h}, \hat{w})$ has been found, we look only in its local neighborhood. A $w \times w$ window of similarity scores is extracted around the peak, a temperature-scaled softmax is applied, and the final prediction is computed as the expected location:

$$\hat{\mathbf{p}} = \sum_{(i,j) \in \mathcal{W}} \operatorname{softmax} \left(\frac{s[\mathcal{W}]}{\tau_s} \right)_{(i,j)} \cdot (i, j),$$

where $s[\mathcal{W}]$ are the cosine-similarity values in the window and τ_s is a tunable temperature. The result is a continuous sub-patch prediction, obtained without changing the backbone weights. We sweep $\tau_s \in \{0.001, 0.005, 0.01, 0.05\}$ and $w \in \{3, 5, 7, 11\}$ on the validation split and report the best configuration on the test split.

3.5. Cross-Dataset Generalization (Stage 4)

To test transfer, we evaluate each fine-tuned backbone on PF-Pascal and PF-Willow *without any PF-specific training*. PF-Pascal covers 20 Pascal VOC categories (~ 1351 pairs), while PF-Willow follows the Proposal Flow benchmark layout. The normalization also changes: SPair-71k uses the target bounding box, whereas the PF benchmarks

use $\alpha \times \max(H, W)$ of the target image. We compare three conditions: (i) frozen argmax, (ii) fine-tuned argmax, and (iii) fine-tuned soft-argmax using the Stage 3 hyperparameters.

4. Experimental Setup

Dataset. SPair-71k [7] contains 70,958 image pairs from 18 Pascal VOC categories. Its test split includes 88,328 keypoint annotations. Throughout the experiments, we use the official `trn/val/test` splits.

Backbones. All backbones use publicly available pretrained weights: DINOv2 ViT-L/14 (`dinov2_vitl14_pretrain.pth`), DINOv3 ViT-B/16 (`dinov3_vitb16_pretrain_lvd1689m.pth`), and SAM ViT-B (`sam_vit_b.01ec64.pth`).

Metric. For SPair-71k, PCK@ α measures the fraction of predicted keypoints that fall within $\alpha \times \max(b_w, b_h)$ of the ground truth, where b_w and b_h denote the width and height of the target bounding box. For PF-Pascal and PF-Willow, we follow the Proposal Flow convention and normalize by $\alpha \times \max(H, W)$ of the target image. We report $\alpha \in \{0.05, 0.1, 0.15, 0.2\}$.

Implementation. All experiments are run in Google Colab on a single GPU. Feature tensors are cached to disk in float16, which keeps repeated evaluation passes manageable.

5. Results

5.1. Frozen Baseline (Stage 1)

Table 1 reports test-split PCK for the three frozen backbones. DINOv2 ViT-L/14 is the strongest baseline, reaching PCK@0.1 = 55.48%, with DINOv3 ViT-B/16 close behind at 53.57%. SAM ViT-B behaves quite differently. Even with its high-resolution feature map, it reaches only 24.42%, which suggests that features optimized for segmentation do not automatically transfer to keypoint-level semantic matching in a zero-shot setting.

Table 1. SPair-71k test, frozen backbone baseline (Stage 1, argmax). 88,328 keypoints, 12,234 image pairs.

Model	@0.05	@0.1	@0.15	@0.2
DINOv2 ViT-L/14	38.57	55.48	64.62	71.05
DINOv3 ViT-B/16	36.66	53.57	62.69	68.37
SAM ViT-B	15.65	24.42	31.75	38.15

5.2. Fine-Tuning Ablation (Stage 2)

Table 2 gives the full 3×3 fine-tuning grid. The pattern is not uniform across backbones, and several points are worth noting.

Table 2. SPair-71k test, fine-tuned backbones (Stage 2, argmax). All runs use $\text{lr}=5 \times 10^{-6}$, 3 epochs.

Model	N	@0.05	@0.1	@0.15	@0.2
DINOv2 ViT-L/14	1	56.64	71.52	77.73	81.65
	2	50.57	65.63	72.63	76.89
	4	58.12	72.85	78.87	82.29
DINOv3 ViT-B/16	1	55.69	73.44	80.56	84.82
	2	56.81	74.48	81.33	85.34
	4	50.28	66.44	73.27	77.37
SAM ViT-B	1	31.55	42.33	50.30	56.36
	2	33.03	44.07	51.88	58.06
	4	29.63	40.30	47.93	53.85

Fine-tuning gives large gains for every backbone. DINOv2 improves most when $N=4$ (+17.37 pp at PCK@0.1), whereas DINOv3 and SAM both peak at $N=2$, gaining +20.91 pp and +19.66 pp respectively. The DINOv3 $N=4$ run is especially informative: it performs worse than $N=2$, which suggests that smaller ViT-B models can become less stable, or simply overfit, when too many deeper blocks are unfrozen. SAM benefits strongly in relative terms, but it still remains far below the DINOv2/DINOv3 range, consistent with the mismatch between segmentation pretraining and semantic correspondence.

5.3. Window Soft-Argmax (Stage 3)

Table 3 compares standard argmax decoding with soft-argmax decoding using the best validation hyperparameters for each backbone. All three models improve. For DINOv2 and DINOv3, the best setting uses temperature $\tau_s=0.05$ and window size $w=7$; for SAM, the optimal window is larger, $w=11$, which is consistent with its 64×64 feature grid.

Table 3. SPair-71k test, Stage 3 argmax vs. soft-argmax. Best- N checkpoint used per backbone.

Model	Mode	@0.05	@0.1	@0.15	@0.2
DINOv2 ($N=4$)	argmax	58.12	72.85	78.87	82.29
	soft-argmax	59.31	73.49	79.18	82.54
DINOv3 ($N=2$)	argmax	56.81	74.48	81.33	85.34
	soft-argmax	59.27	76.41	82.76	86.01
SAM ($N=2$)	argmax	33.03	44.08	51.88	58.07
	soft-argmax	33.70	46.02	53.13	58.93

The gains are concentrated at the tighter thresholds, especially PCK@0.05 and PCK@0.1. This is exactly where sub-

patch localization should matter: the smaller the allowed error radius, the more valuable it becomes to move beyond the discrete patch center. DINOv3 benefits more than DINOv2 (+2.46 pp at @0.05 and +1.93 pp at @0.1), perhaps because its ViT-B grid is coarser (32×32) and therefore leaves more room for useful sub-cell interpolation. Figure 1 makes this threshold-dependent behavior explicit: soft-argmax stays close to argmax at loose thresholds, but it opens a clearer gap when the evaluation radius is stricter.

5.4. Per-Image and Per-Category Analysis

The uploaded per-image records are most useful not as supplementary raw files, but as a way to derive a compact diagnostic summary of Stage 3 behavior. Each record corresponds to one SPair-71k source–target pair and reports the number of evaluated keypoints together with PCK at four thresholds. We aggregate them into Table 4: the PCK columns are keypoint-weighted across all 12,234 test pairs, and the gain column isolates the improvement obtained by replacing discrete argmax decoding with window soft-argmax for the same fine-tuned backbone.

The table makes the effect of the decoder more precise than the aggregate headline alone. Soft-argmax improves every backbone at every threshold, but the pattern is not identical across models. DINOv3 receives the strongest overall refinement, gaining +1.93 pp at PCK@0.1 and reaching the best score across the four reported thresholds. SAM gains almost the same amount at PCK@0.1 (+1.95 pp), although its absolute performance remains far below the DINO backbones, which shows that sub-patch refinement cannot compensate for weaker semantic descriptors. DINOv2 changes more modestly, suggesting that many of its recoverable matches were already close to the argmax location. Overall, the per-image aggregation supports the same conclusion as the threshold curves: window soft-argmax is a reliable refinement step, while the main separation between methods still comes from the quality of the learned backbone representation.

Table 5 then reports category-level mean PCK@0.1 for all

Table 4. SPair-71k per-image aggregation for the Stage 3 best checkpoints. All rows use 12,234 image pairs and 88,328 evaluated keypoints. PCK values are percentages; $\Delta@0.1$ is the soft-argmax gain over argmax for the same backbone.

Backbone	Dec.	@0.05	@0.1	@0.15	@0.2	$\Delta@0.1$
DINOv2	arg.	58.12	72.85	78.87	82.29	–
DINOv2	soft	59.31	73.49	79.18	82.54	+0.64
DINOv3	arg.	56.81	74.48	81.33	85.34	–
DINOv3	soft	59.27	76.41	82.76	86.01	+1.93
SAM	arg.	33.03	44.08	51.89	58.07	–
SAM	soft	33.70	46.02	53.13	58.93	+1.95

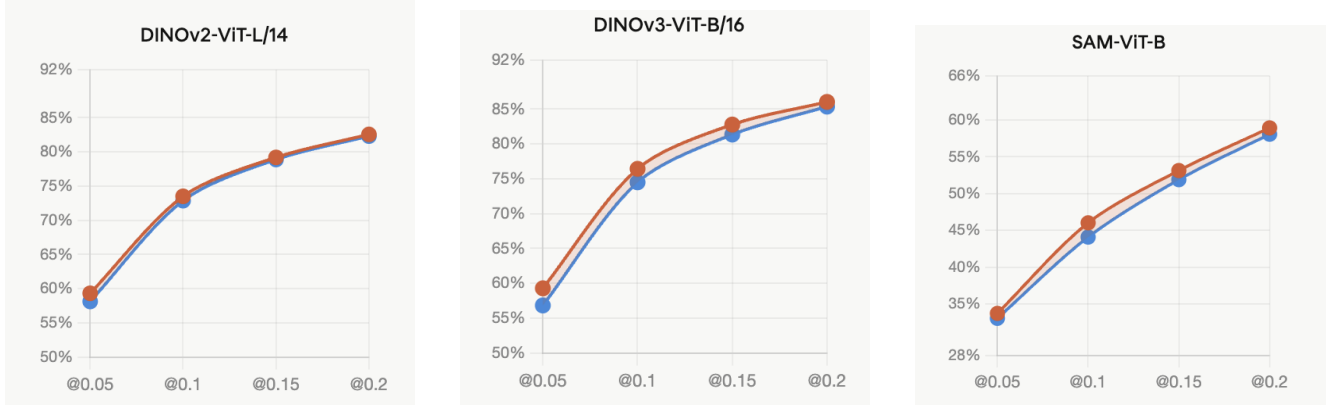


Figure 1. SPair-71k PCK curves across evaluation thresholds for the best fine-tuned checkpoint of each backbone. Blue denotes standard argmax decoding and orange denotes window soft-argmax decoding. The soft-argmax curve is consistently above argmax, with the largest practical separation appearing at the stricter thresholds where sub-patch localization matters most.

18 Pascal VOC categories using the soft-argmax condition. The rightmost column identifies the best model in each row.

Table 5. SPair-71k, per-category mean PCK@0.1 (%) with soft-argmax (Stage 3 best configuration). Bold = best per category.

Category	DINOv2	DINOv3	SAM	Best
aeroplane	84.64	76.04	43.66	D2
bicycle	64.91	63.57	28.96	D2
bird	86.85	87.66	66.86	D3
boat	56.35	56.49	32.70	D3
bottle	55.42	63.35	35.52	D3
bus	81.14	73.10	39.20	D2
car	70.01	68.85	31.90	D2
cat	90.06	85.86	62.47	D2
chair	59.67	50.57	25.60	D2
cow	86.90	85.60	48.29	D2
dog	74.60	75.12	40.47	D3
horse	75.84	75.07	38.99	D2
motorbike	62.70	68.80	31.66	D3
person	68.21	74.26	50.83	D3
pottedplant	44.93	67.08	41.22	D3
sheep	70.04	64.82	32.17	D2
train	79.88	87.55	60.22	D3
tvmonitor	55.97	83.22	44.41	D3
Mean	69.66	72.44	42.01	–

Category-level winners. DINOv2 leads on 10 of the 18 categories, mostly among *animal* classes (cat, cow, bird, horse, sheep) and large rigid objects such as aeroplane, bus, and car. DINOv3 is best on the remaining 8 categories. Its largest margins appear on structured or patterned objects: **tvmonitor** (+27.25 pp over DINOv2), **pottedplant** (+22.15 pp), **train** (+7.67 pp), and **person** (+6.05 pp). SAM does not win any category, though it comes closest to the DINO

models on **bird** (66.86%) and **person** (50.83%). In those cases, its high-resolution (64×64) feature map may partly compensate for weaker semantic abstraction.

Universally difficult categories. Some categories remain difficult regardless of the backbone. **chair** (DINOv2: 59.67%, DINOv3: 50.57%), **boat** (56.35%, 56.49%), and **pottedplant** (44.93%, 67.08%) are the clearest examples. Chairs vary heavily in shape, boats are often occluded or viewed from incompatible angles, and potted plants have irregular, sometimes symmetric keypoint layouts. DINOv3 handles pottedplant much better than DINOv2 (67.08% vs. 44.93%), which points to a real difference in how the two pretraining setups encode fine-grained spatial structure for objects without stable rigid parts.

Perfect and failed pairs. Table 6 summarizes success and failure at the pair level. DINOv3 reaches perfect PCK@0.1 (= 100%) on **26.6%** of test pairs, which is 546 more perfect pairs than DINOv2. It also has the lowest complete-failure rate: only 3.9% of pairs score PCK@0.1 = 0%, compared with 4.8% for DINOv2. SAM’s profile is much weaker. It scores zero on **15.0%** of pairs, indicating that its features often fail to separate fine-grained semantic keypoints well enough for this benchmark.

Table 6. Pair-level PCK@0.1 extremes (12,234 pairs).

Model	PCK@0.1 = 100%		PCK@0.1 = 0%	
	Count	%	Count	%
DINOv2 ViT-L/14	2710	22.2	592	4.8
DINOv3 ViT-B/16	3256	26.6	480	3.9
SAM ViT-B	402	3.3	1838	15.0

5.5. Cross-Dataset Generalization (Stage 4)

Tables 7 and 8 report zero-shot transfer results on PF-Pascal and PF-Willow.

Table 7. PF-Pascal test, PCK@0.1, $\alpha \cdot \max(H, W)$ normalization. No training on PF-Pascal.

Model / Condition	@0.05	@0.1	@0.15	@0.2
DINOv2, frozen	62.49	80.69	88.20	92.01
DINOv2, ft argmax	66.39	83.75	90.55	93.90
DINOv2, ft soft-arg	67.00	84.29	90.92	94.09
DINOv3, frozen	66.45	86.49	93.12	96.09
DINOv3, ft argmax	70.02	85.17	90.81	93.27
DINOv3, ft soft-arg	71.53	86.04	91.17	93.39
SAM, frozen	26.13	38.92	48.49	57.09
SAM, ft argmax	35.93	48.98	58.87	66.79
SAM, ft soft-arg	37.08	49.79	59.15	66.96

Table 8. PF-Willow test, PCK@0.1, macro-category average. No training on PF-Willow.

Model / Condition	@0.05	@0.1	@0.15	@0.2
DINOv2, frozen	52.56	77.33	86.78	91.22
DINOv2, ft argmax	61.89	84.22	91.44	94.22
DINOv2, ft soft-arg	62.22	85.56	91.67	94.33
DINOv3, frozen	62.89	85.67	93.56	96.22
DINOv3, ft argmax	66.67	87.78	94.11	96.67
DINOv3, ft soft-arg	69.00	89.22	94.11	96.67
SAM, frozen	30.11	46.11	54.78	63.56
SAM, ft argmax	37.56	55.00	65.11	72.56
SAM, ft soft-arg	38.33	55.78	65.67	73.00

Several findings emerge. **Transfer is positive for DINOv2 and SAM:** SPair-71k fine-tuning improves both PF-Pascal and PF-Willow performance for these backbones across all reported metrics. **DINOv3 is different on PF-Pascal:** its frozen features are already extremely strong, and fine-tuning slightly lowers PCK@0.1 (86.49% $\rightarrow$ 86.04%). This suggests that DINOv3 pretraining on LVD-1689M may already encode the correspondences most useful for that benchmark. PF-Willow tells a different story, since DINOv3 still gains +3.55 pp at PCK@0.1 with soft-argmax. **SAM improves sharply** even though its absolute scores remain lower: +10.87 pp on PF-Pascal and +9.67 pp on PF-Willow at PCK@0.1. In other words, even a weak semantic backbone can become much more useful once it receives a task-specific contrastive alignment signal.

5.6. Summary of Gains

Figure 2 places the SPair-71k progression side by side. The main jump comes from fine-tuning, which adds roughly +17–

21 pp at PCK@0.1. Soft-argmax then contributes a smaller but consistent additional gain of +0.6–1.9 pp.

6. Discussion

Optimal N depends on capacity. DINOv2 ViT-L/14 benefits most from unfreezing $N=4$ blocks, while both ViT-B backbones peak at $N=2$. This fits a simple interpretation: larger models have more adaptable capacity in their deeper layers, whereas smaller models are easier to disturb if too many blocks are updated at once.

Soft-argmax gains tighter thresholds most. Soft-argmax helps most at PCK@0.05, less at PCK@0.1, and only modestly at PCK@0.15–0.2. That behavior is expected. Subpatch interpolation matters when the allowed error radius is close to, or smaller than, one patch width. Once the threshold is loose enough, the discrete argmax location is often already sufficient.

Complementary categorical strengths. Table 5 shows that DINOv2 and DINOv3 are *complementary*, not simply better or worse versions of one another. DINOv2 leads on 10 of 18 categories, especially animals such as cat, cow, sheep, and horse, along with large rigid structures such as aeroplane and bus. DINOv3 is stronger on more structured, often man-made objects. The tvmonitor gap is striking: 83.22% for DINOv3 versus 55.97% for DINOv2, a +27.25 pp difference. Pottedplant (67.08% vs. 44.93%) and train (87.55% vs. 79.88%) show a similar tendency. These results suggest that DINOv3 pretraining may encode stronger texture-level and structural part-level cues, while DINOv2’s larger ViT-L capacity remains valuable for deformable, high-variation categories. Combining the two backbones, either by ensembling or feature fusion, is therefore a natural direction for future work.

Hard categories reveal benchmark structure. Chair and boat remain difficult for every model (DINOv2: 59.67% and 56.35%; DINOv3: 50.57% and 56.49%). This points less to a single backbone weakness and more to the structure of SPair-71k itself. Chairs have extreme intra-class variation and few consistently distinctive parts. Boats are often seen from very different viewpoints, sometimes with ambiguous symmetry. These categories likely need geometric reasoning beyond direct feature-level matching.

SAM as a correspondence backbone. SAM is trained to predict multi-scale segmentation masks from sparse prompts, not to produce discriminative patch-level descriptors across object instances. Its low frozen baseline (24.42%), high complete-failure rate (15.0% of pairs at PCK@0.1 = 0), and absence from the per-category leaderboard all reflect

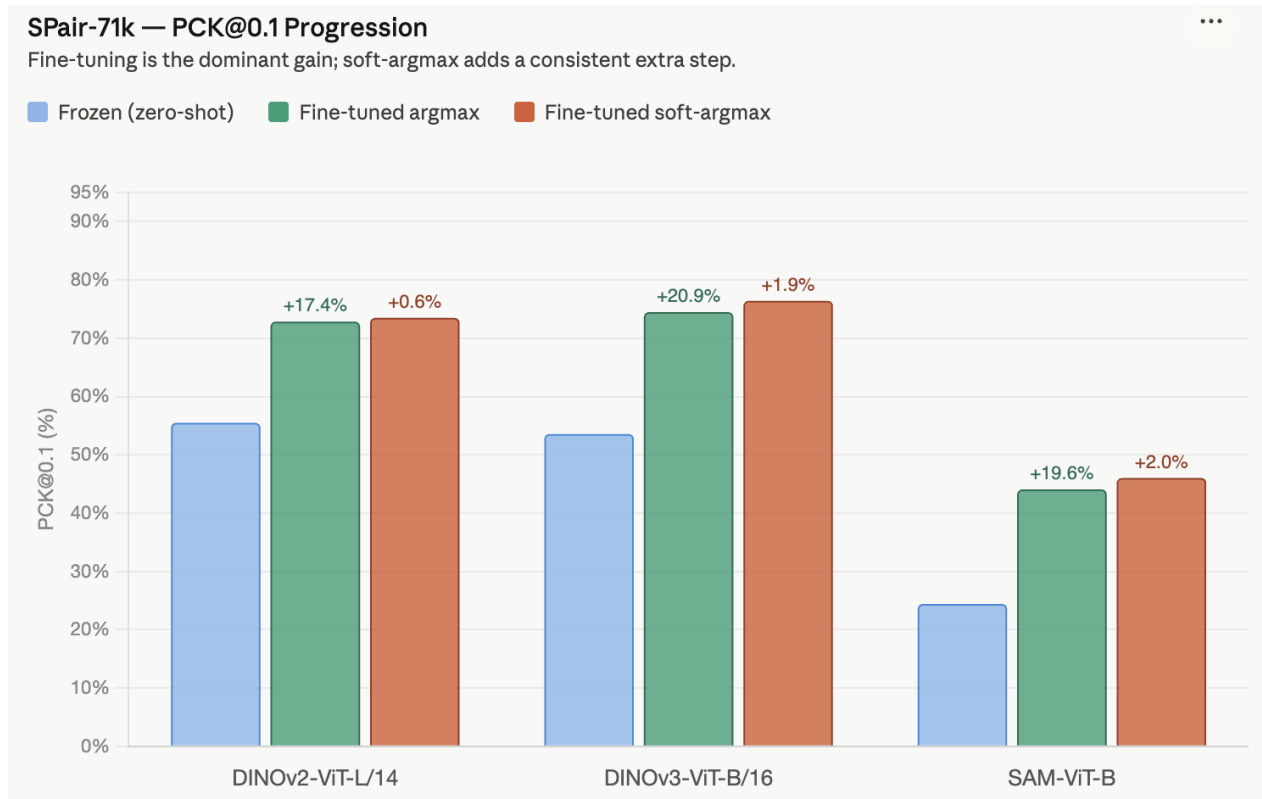


Figure 2. SPair-71k PCK@0.1 progression from frozen zero-shot matching to InfoNCE fine-tuning and post-hoc window soft-argmax decoding. The figure highlights that fine-tuning produces the dominant improvement for all three backbones, while soft-argmax adds a smaller but consistent localization gain on top of the best fine-tuned checkpoint.

this domain gap. Still, the large gains after InfoNCE fine-tuning (+20 pp on SPair-71k; +10.87 pp on PF-Pascal) show that the architecture is not inherently unsuitable. It needs a representation-level alignment signal.

DINOv3 generalization. DINOv3 ViT-B/16 reaches higher fine-tuned PCK on SPair-71k than DINOv2 ViT-L/14, even though it is the smaller model. It also has the best perfect-pair rate (26.6% at PCK@0.1 = 100%) and the lowest failure rate (3.9%). Its pretraining on the larger LVD-1689M dataset, together with the revised DINO objective, may produce features that align particularly well with semantic matching. For compute-constrained settings, this makes DINOv3 a strong practical choice.

7. Conclusion

We presented a four-stage pipeline for dense semantic correspondence with self-supervised ViT backbones. Beginning from a frozen cosine-similarity baseline, we showed that light InfoNCE fine-tuning of the last N transformer blocks produces large SPair-71k gains, roughly +17 to +21 pp at PCK@0.1. A post-hoc window soft-argmax decoder

then adds a smaller but reliable improvement at tight thresholds (+0.6 to +1.9 pp), without requiring further training. The per-image analysis over all 12,234 test pairs shows that DINOv2 and DINOv3 have complementary strengths: DINOv2 leads on 10 of 18 categories, mainly deformable animals and rigid vehicles, while DINOv3 is stronger on structured objects, including a +27 pp advantage on tvmonitor. Both DINO models keep complete-failure rates below 5%, whereas SAM scores zero on 15% of pairs in the frozen setting. Finally, the SPair-71k fine-tuned models transfer well to PF-Pascal and PF-Willow without additional training, with DINOv3 reaching 89.22% PCK@0.1 on PF-Willow.

These findings suggest several next steps. One is to combine DINOv2-scale capacity with DINOv3-style pretraining data. Another is to study how batch size and training duration affect fine-tuning stability, especially when more blocks are unfrozen. A third direction is to add geometric consistency terms, such as cycle consistency or flow regularization, to the InfoNCE loss so that ambiguous matches are suppressed more directly.

References

- [1] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021. [2](#)
- [2] Seokju Cho, Sunghwan Kim, Dongbo Han, Suha Han, and Tae Hyun Park. CATs: Cost aggregation transformers for visual correspondence. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. [2](#)
- [3] Bumsub Ham, Minsu Cho, Cordelia Schmid, and Jean Ponce. Proposal flow. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016. [2](#)
- [4] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C. Berg, Wan-Yen Lo, Piotr Dollár, and Ross Girshick. Segment anything. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023. [2](#)
- [5] David G. Lowe. Distinctive image features from scale-invariant keypoints. pages 91–110, 2004. [1](#)
- [6] Juhong Min, Jongmin Lee, Jean Ponce, and Minsu Cho. Hyperpixel flow: Semantic correspondence with multi-layer neural features. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019. [2](#)
- [7] Juhong Min, Jongmin Lee, Jean Ponce, and Minsu Cho. SPair-71k: A large-scale benchmark for semantic correspondence. In *arXiv preprint arXiv:1908.10543*, 2019. [2](#), [3](#)
- [8] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, et al. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024. [2](#)
- [9] Luming Tang, Menglin Jia, Qianqian Wang, Cheng Perng Phoo, and Bharath Hariharan. Emergent correspondence from image diffusion. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. [2](#)
- [10] Junyi Zhang, Charles Herrmann, Junhwa Hur, Luisa Polania Cabral, Varun Jampani, Deqing Sun, and Ming-Hsuan Yang. A tale of two features: Stable diffusion complements DINO for zero-shot semantic correspondence. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. [2](#)